

IN4MATX 285:

Interactive Technology Studio

**Programming: Loops and
Functions in JavaScript**

Today's goals

By the end of today, you should be able to...

- Explain why we need tools to help us code repetitive tasks more efficiently
- Loop over things in JavaScript
- Call functions with JavaScript that do useful operations
- Write functions to help avoid repetition

Efficiency

- You can actually do a lot of things with just the coding tools we've learned so far!
 - Store and update values
 - Choose what should be stored based on conditions
- But, you will very quickly run into *efficiency* problems
- (And next week, we'll learn how to use these coding tools to edit interfaces)

Efficiency

- Imagine you needed to display 4 image URLs in a gallery
- You could create a variable for each one, for example. Not too bad!

```
let image1URL = 'landscape.jpg';  
let image2URL = 'turtle.jpg';  
let image3URL = 'clouds.jpg';  
let image4URL = 'trees.jpg';
```



Efficiency

- You take some more pictures, and now you need to display 20 image URLs
- Your code is getting longer, even though your goal hasn't changed much...

```
let image1URL = 'landscape.jpg';  
let image2URL = 'turtle.jpg';  
let image3URL = 'clouds.jpg';  
let image4URL = 'trees.jpg';  
//...  
let image20URL = 'beach.jpg';
```



Efficiency

- Now, you need to implement infinite scroll. Uh oh...
- You wrote some code to convert each JPG to PNG, but have to copy-paste it for each image...
- **There has to be a better way!**



Efficiency

- Enter: *loops* and *functions*
- Okay, these don't totally solve the problems stated.
- But they help us get a lot closer.
- Data structures (coming up later in the quarter) are the last piece

Loops

Loops

- Thus far, every line of code is executing sequentially
- Loops allow you to *repeat* lines of code, while changing variables every time

Loops

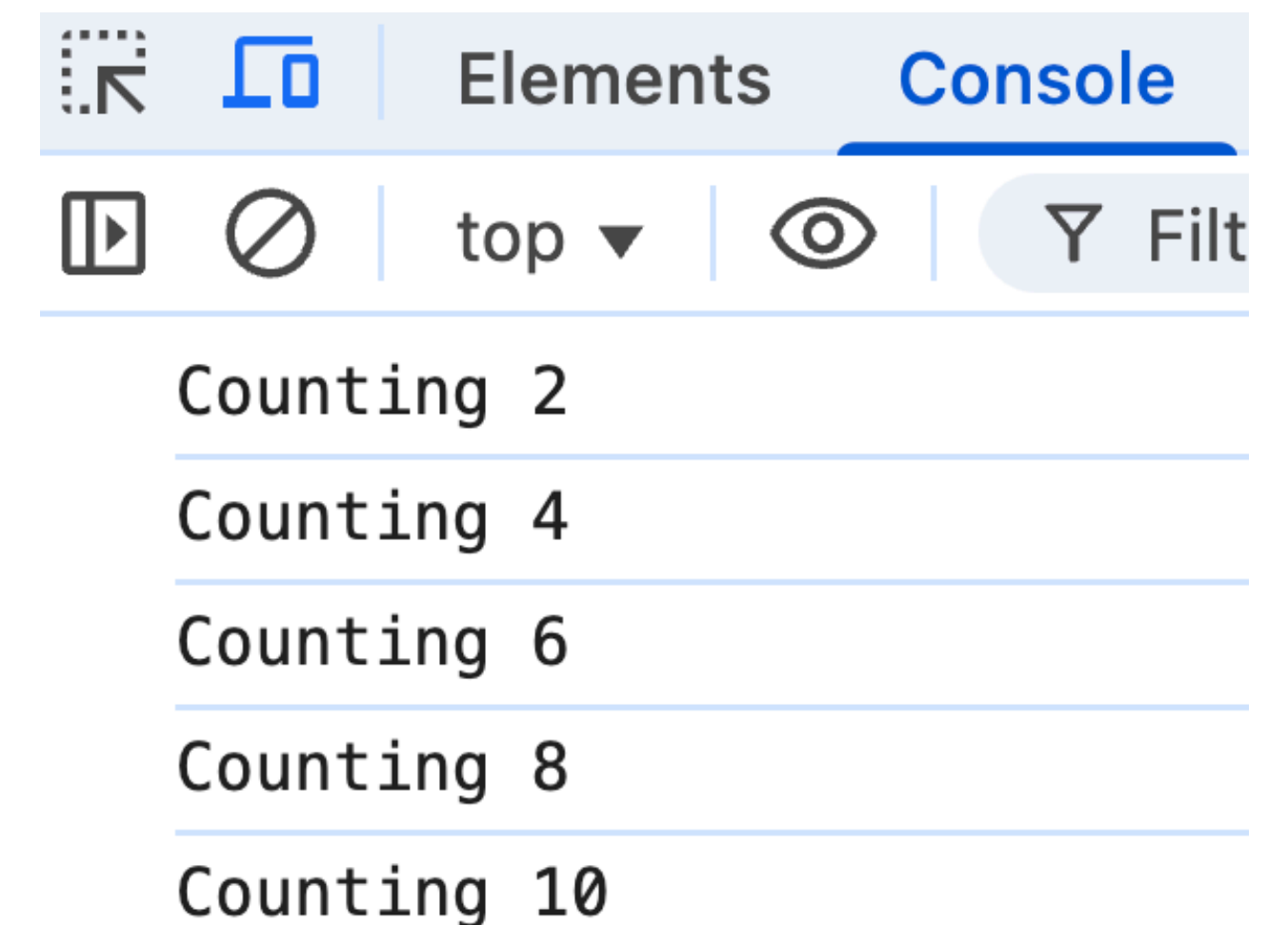
//Let's count to 10 by twos!

Initialize End condition Advancement



➔

```
for(let i = 1; i <= 5; i = i + 1) {  
  let iTimesTwo = i * 2;  
  console.log("Counting " + iTimesTwo);  
}
```



Loops

```
//Let's count to 10 by twos!  
for(let i = 1; i <= 5; i = i + 1) {  
    let iTimesTwo = i * 2;  
    console.log("Counting " + iTimesTwo);  
}
```

```
//Does the same thing  
for(let i = 2; i <= 10; i = i + 2) {  
    console.log("Counting " + i);  
}
```


Loops



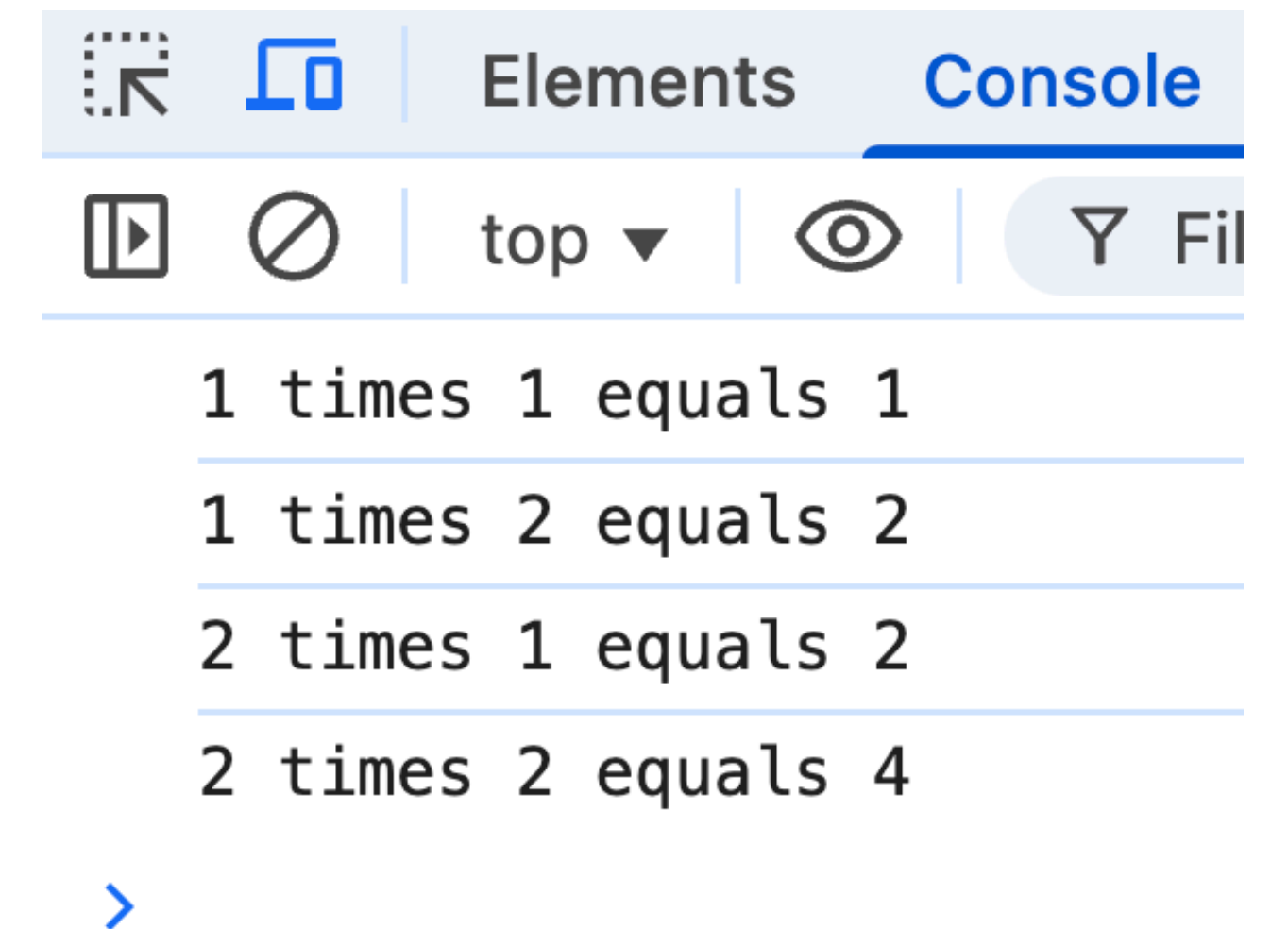
1 squared is 1	<u>solution1.js:3</u>
2 squared is 4	<u>solution1.js:3</u>
3 squared is 9	<u>solution1.js:3</u>
4 squared is 16	<u>solution1.js:3</u>
5 squared is 25	<u>solution1.js:3</u>
6 squared is 36	<u>solution1.js:3</u>
7 squared is 49	<u>solution1.js:3</u>
8 squared is 64	<u>solution1.js:3</u>
9 squared is 81	<u>solution1.js:3</u>
10 squared is 100	<u>solution1.js:3</u>

Loops

- Loops can be nested

//Two by two times table

```
for(let i = 1; i <= 2; i = i + 1) {  
    for(let j = 1; j <= 2; j = j + 1) {  
        console.log(i + ' times ' + j + ' equals ' + i*j);  
    }  
}
```



Loops



1	2	3	4	5	<u>solution2.js:14</u>
2	4	6	8	10	<u>solution2.js:14</u>
3	6	9	12	15	<u>solution2.js:14</u>
4	8	12	16	20	<u>solution2.js:14</u>
5	10	15	20	25	<u>solution2.js:14</u>

Loops



1	solution3.js:4
3	solution3.js:4
5	solution3.js:4
7	solution3.js:4
9	solution3.js:4
11	solution3.js:4
13	solution3.js:4
15	solution3.js:4
17	solution3.js:4
19	solution3.js:4
1	solution3.js:8
3	solution3.js:8
5	solution3.js:8
7	solution3.js:8
9	solution3.js:8
11	solution3.js:8
13	solution3.js:8
15	solution3.js:8
17	solution3.js:8
19	solution3.js:8

So, what if we want to repeat something in our code?

Functions

- Functions allow you to write code which can be used many times
- You can use the same code with different *arguments* to produce different results
- Code libraries often provide functions with useful tools for programmers

Calling Functions

- Refer to the function's *name*
- Pass *argument(s)* in parentheses
- Sometimes *returns* a value
- Often requires looking up the reference to figure out how to use
- Example: isFinite

Calling Functions

- Example: `isFinite`
- Returns `true` if the argument is a finite number (e.g., between $-\infty$ and ∞), and `false` otherwise
- You can, and often do, call functions within other functions

<code>console.log(isFinite(285));</code>	<code>true</code>
<code>console.log(isFinite('285'));</code>	<code>true</code>
<code>console.log(isFinite('IN4MATX 285'));</code>	<code>false</code>

Calling Functions

- Functions often have dots in their names (e.g., `console.log()`)
- Technically, the `log` function exists only within the context of `console`, which is an *Object*
- You can also call functions *on* variables, which are themselves Objects
- e.g., `textString.toLowerCase()`
- But, we'll largely ignore Objects in this class

Calling Functions



<code>in4matx 285</code>	<u>solution4.js:2</u>
<code>IN4MATX 285</code>	<u>solution4.js:6</u>
<code>285</code>	<u>solution4.js:11</u>
<code>true</code>	<u>solution4.js:12</u>

Writing Functions

- But, you can also write your own functions
- Essentially, you're making a library for yourself

Writing Functions

Function *name* Function *argument(s)*

↓ ↓

```
function toCelsius(fahrenheit) {  
    return (5/9) * (fahrenheit - 32);  
}
```

↑
Value to *return*

```
let tempF = 77;  
console.log(tempF + ' in Celsius is ' + toCelsius(tempF));  
tempF = 32;  
console.log(tempF + ' in Celsius is ' + toCelsius(tempF));
```

77 in Celsius is 25

32 in Celsius is 0

Writing Functions



32 in Fahrenheit is 273.15K

25 in Celsius is 298.15K

99 in Red Balloons is undefinedK

Functions

- In Javascript, all parameters are optional

```
function sayHello(name)
```

```
{
```

```
    return "Hello, "+name;
```

```
}
```

```
//expected; parameter is assigned a value
```

```
sayHello("In4MATX 133"); //"Hello, IN4MATX 133"
```

```
//parameter not assigned value (left undefined)
```

```
sayHello(); //"Hello, undefined"
```

```
//extra parameters (values) are not assigned
```

```
//to variables, so are ignored
```

```
sayHello("IN4MATX", "133"); //"Hello, IN4MATX"
```

Other concepts

- There are tons of basic programming concepts I am skipping over
 - While loops
 - Different modifiers (++ , += , % , /)
 - String manipulation
 - Objects
- A lot of these aim to make code more efficient or more concise
- Ask on Slack, or search online, if you need to understand how they work

Today's goals

By the end of today, you should be able to...

- Explain why we need tools to help us code repetitive tasks more efficiently
- Loop over things in JavaScript
- Call functions with JavaScript that do useful operations
- Write functions to help avoid repetition

IN4MATX 285:

Interactive Technology Studio

**Programming: Loops and
Functions in JavaScript**

Additional slides

Let, Var, and Const

- Both `let` and `var` can be used to create a new variable
- `const` can be used to declare a variable that won't be changed
- There are some differences in how variables created with each are visible across your code
 - But these differences are pretty subtle and advanced, and largely won't matter for the code that we're creating in this class
- You might see some examples which use `var` to make new variables

null, undefined, and NaN

- `null`: a nonexistent object
 - Therefore it is an object, just uninitialized

```
var nullObj = null;
```

```
console.log(typeof nullObj); //object
if(!nullObj) {
  console.log("It's falsy");
}
//but it's not equal to false
console.log(nullObj == false); //false
```

null, undefined, and NaN

- `undefined`: an undefined primitive value
 - Therefore it's a primitive value, like a number or a string

```
var undefinedObj;
```

```
console.log(undefinedObj); //undefined
console.log(typeof undefinedObj); //undefined
if(!undefinedObj) {
  console.log("It's falsy");
}
//but it's not equal to false
console.log(undefinedObj == false); //false
```

<https://codeburst.io/understanding-null-undefined-and-nan-b603cb74b44c>

null, undefined, and NaN

- NaN: Not a Number
 - Will be the result of any computation on an `undefined` value
 - Or any other impossible computation
 - But it's type is a number (despite the name)

```
console.log('12' - 5); // 7
console.log('word' - 5); // NaN
console.log(undefined * 3); // NaN
console.log(typeof NaN); // number
if(NaN) {
  console.log("It's not falsy!");
}
```

<https://codeburst.io/understanding-null-undefined-and-nan-b603cb74b44c>